

Operating Systems – Design Patterns

Sistemas de Computadores

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- Introduction
- Kinds of operating systems
- Separation of policy and mechanism
- Design patterns

Introduction

- **As seen in today's lecture, modern processors provide essential hardware mechanisms that enable an OS to enforce isolation and control execution**
 - Dual-mode operation (user mode and kernel mode)
 - Memory protection through virtual memory and the MMU
 - Interrupts, exceptions, and system calls
 - Hardware timers for preemptive multitasking
- **On their own, however, these mechanisms do not define how an OS is organized internally**
 - They simply provide the tools that make protection, abstraction, and controlled resource sharing possible

Introduction

- **In this document, we shift our focus from hardware support to internal kernel structure**
 - Specifically, we explore how OSes are designed to use these mechanisms to implement common services

- **Different OSes adopt different structural approaches**
 - Some place nearly all functionality inside the kernel
 - Others minimize the kernel and move services to user space
 - Some attempt to combine both strategies

Kinds of operating systems

- **OSes are used in many different contexts, for fulfilling many different purposes**
 - Despite all those differences, they all share the same underlying principles

- **Mainframe and server OSes must maximize utilization of hardware**
 - OS doesn't require a graphical user interface
 - Rather, must support very efficient handling of I/O, and possibly scheduling of many processes

Kinds of operating systems

- **Personal computers must be easy to use and responsive to user input**
 - Maximizing hardware utilization is less important – responding to user interaction is top priority!
 - Much more code is devoted to making the computer easy to use
 - Important to provide a simplified, user-friendly user interface

- **OSes for mobile devices and tablets have several challenging, often conflicting constraints**
 - Must be responsive and user-friendly, like PC operating systems
 - But must also try to maximize battery life through careful hardware resource management

Kinds of operating systems

- **Smartphones must support download, installation, and execution of wide range of applications**
 - But basic device capabilities (e.g., voice calls, SMS) must also be rock-solid reliable
 - Must support intermittent connectivity, particularly when programs are using that connectivity!

- **By far the most common kind of computer now is the embedded computer**
 - In your microwave oven, printer, Wi-Fi router, media player, controlling your car engine, your point-and-shoot camera, ...

Kinds of operating systems

■ Embedded OSES tend to have very limited capabilities

- Systems tend to support a specific, fixed set of tasks
- Systems aren't designed to run arbitrary programs on them

■ Can still include a variety of basic OS capabilities

- Basic thread-management and scheduling support
- Basic memory management capabilities
- Support for software upgrades
- Support for peripherals like flash cards, USB drives, networking, ...

Kinds of operating systems

■ Real-time OSeS focus on completing tasks by a specific deadline

- Most general-purpose OSeS provide soft real-time support
 - e.g., for media playback
 - Not considered a system failure if the OS misses a deadline from time to time (e.g., your media playback just sounds choppy)
- Some real-time OSeS provide hard real-time guarantees
 - If the OS misses a deadline, this is considered a fatal error!

■ Different operating system contexts impose different design constraints

- These constraints strongly influence how the kernel itself is structured

Separation of Policy and Mechanism

- One of the most important principles in operating system design is the **separation of policy and mechanism**
 - This distinction helps structure complex systems and makes operating systems more flexible, maintainable, and adaptable over time
- A mechanism defines how something is done
 - It provides the low-level capability or procedure needed to perform an operation
- They are the building blocks provided by the kernel
 - Often relying directly on hardware features discussed in the previous lecture (privilege levels, MMU, traps, timers, etc.)

Separation of Policy and Mechanism

■ Mechanisms are typically:

- Closely tied to hardware support
- Relatively stable over time
- Performance-critical
- General-purpose

■ Mechanisms answer questions like:

- How do we switch from one process to another?
- How do we move a page from memory to disk?
- How do two processes exchange data safely?

Separation of Policy and Mechanism

- **A policy defines what should be done**

- It determines which choice the system makes among several valid possibilities

- **Policies are typically:**

- High-level decisions
- Likely to change over time
- Configurable or tunable
- Workload-dependent

Separation of Policy and Mechanism

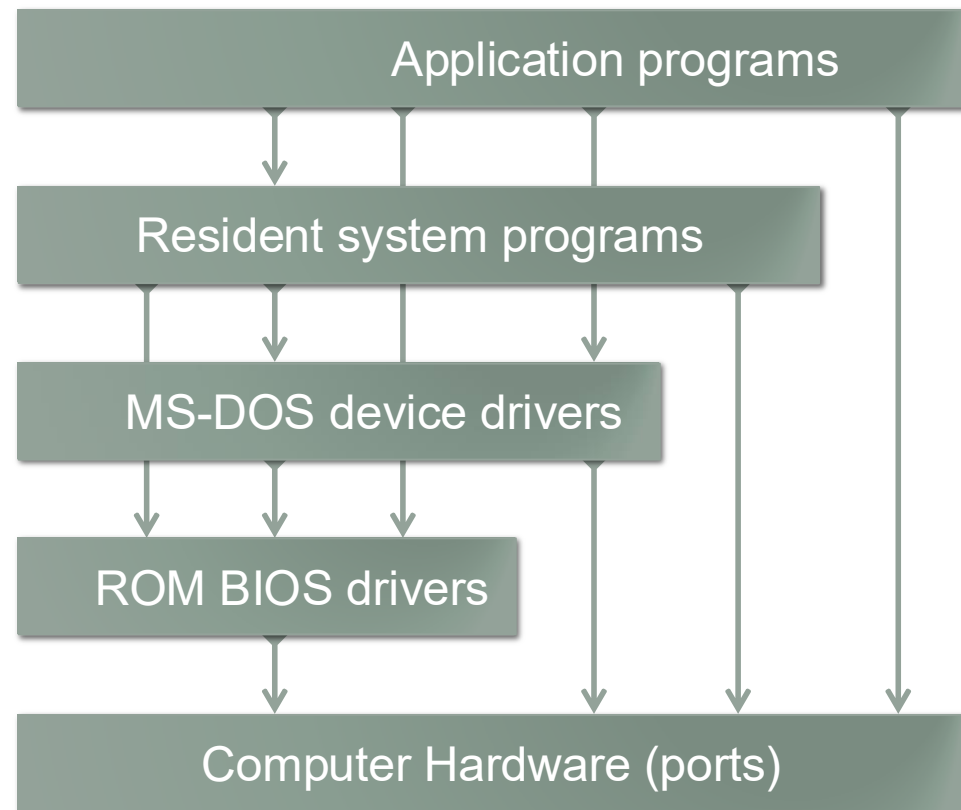
- **Policies answer questions like:**
 - Should we prioritize interactive tasks?
 - Should we favor fairness or throughput?
 - Should we optimize for latency or battery life?
- **A useful rule of thumb in OS design is:**
 - Keep mechanisms minimal, stable, and efficient
 - Keep policies flexible, replaceable, and configurable
- **This principle will reappear throughout the course**
 - Particularly in scheduling, memory management, and file system design

Simple structure

- **OS code is not cleanly divided into modules**
 - Every part of the OS can access every other part of the OS
 - (Often, everything can also access the computer hardware)
- **Usually happens when OS starts out as a simple, limited system and then grows well beyond its original scope**
- **Problems:**
 - Highly susceptible to both operating system and application bugs
 - Often very easy for malicious programs to compromise the OS
 - Tends to be very difficult to extend the OS' functionality

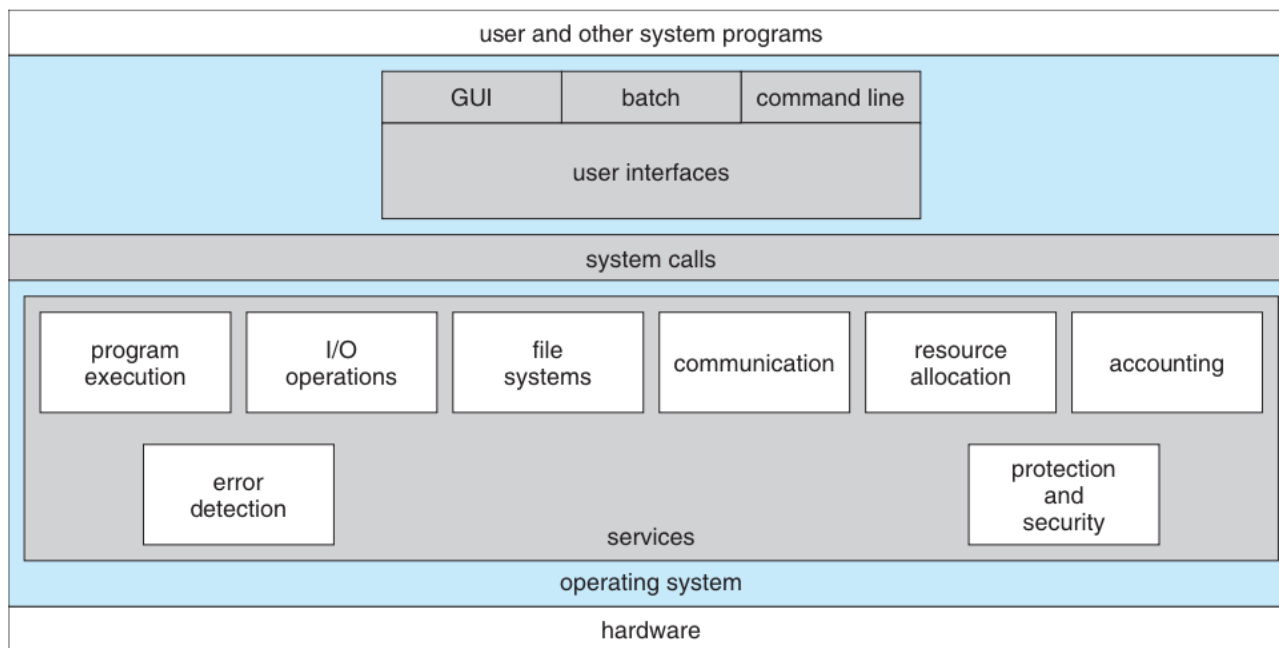
Simple structure: MS-DOS

- **Written for Intel 8086/8088 processors, which had no protected mode execution**
 - Everything could access everything else in the system (and often did!)
- **Apps and resident programs could control hardware**
 - e.g., computer games with specialized graphics mode
 - e.g., extended-memory libs
- **Malicious programs could directly manipulate the OS**
 - e.g., stealth viruses would intercept DOS `int 21` trap to make themselves invisible



Monolithic kernels

- **Initial versions of UNIX were slightly more structured**
 - OS was divided into kernel and system applications
- **A large part of OS functionality is built into the kernel itself**
 - In this design, the kernel consists of a single, large executable



Monolithic kernels

- **OS is divided into kernel and system applications**

- Kernel runs in protected mode
- So, applications can't compromise system easily

- **However, still susceptible to severe bugs, OS crashes**

- Their security can be compromised by a bug or vulnerability in any service since all services run in kernel mode

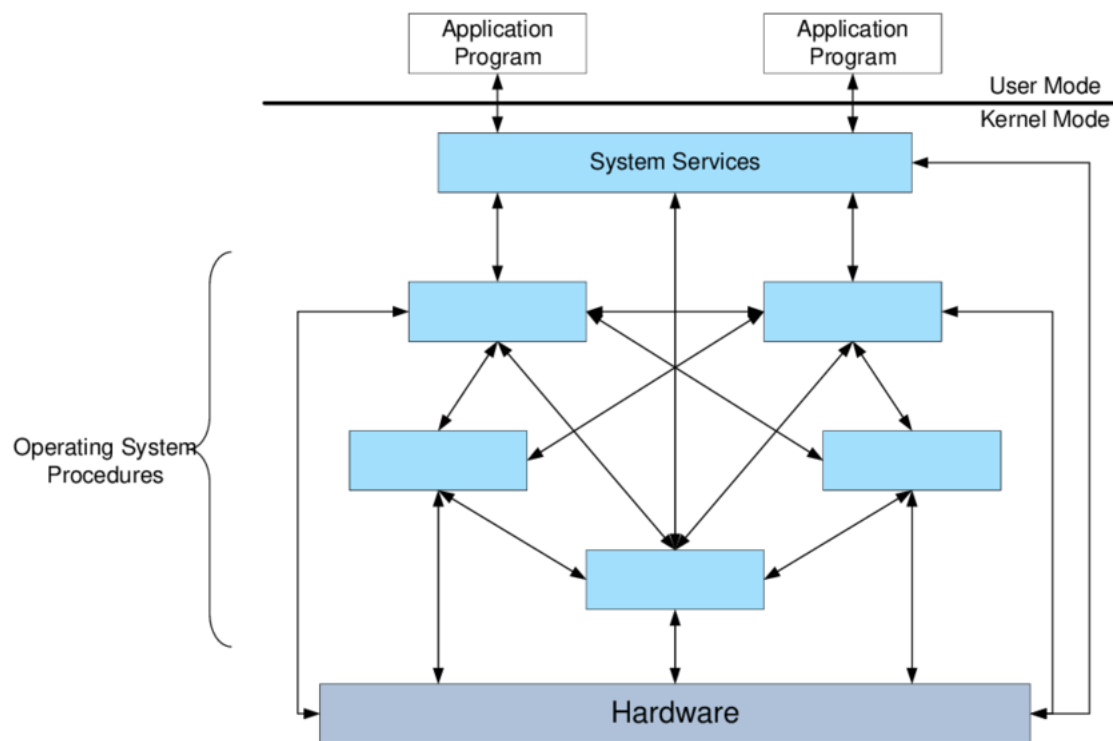
- **Monolithic kernels also have extensibility issues**

- e.g., want to support a wide range of devices, filesystems, etc.
- They lack modularity, making it difficult to update or replace individual services without rebuilding the entire kernel
- Kernel becomes huge, because it must include compiled-in support for all supported devices, filesystems, etc.

Monolithic kernels

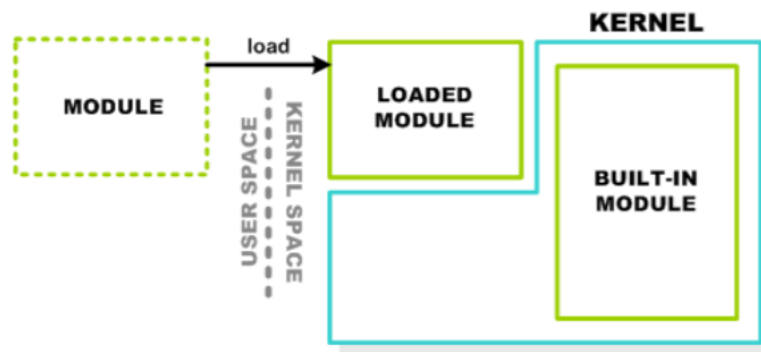
■ Big benefit: it's fast!

- Easy for kernel operations to directly access whatever state or functions they need
- Many modern OSes include monolithic aspects in their implementations



Modular kernels

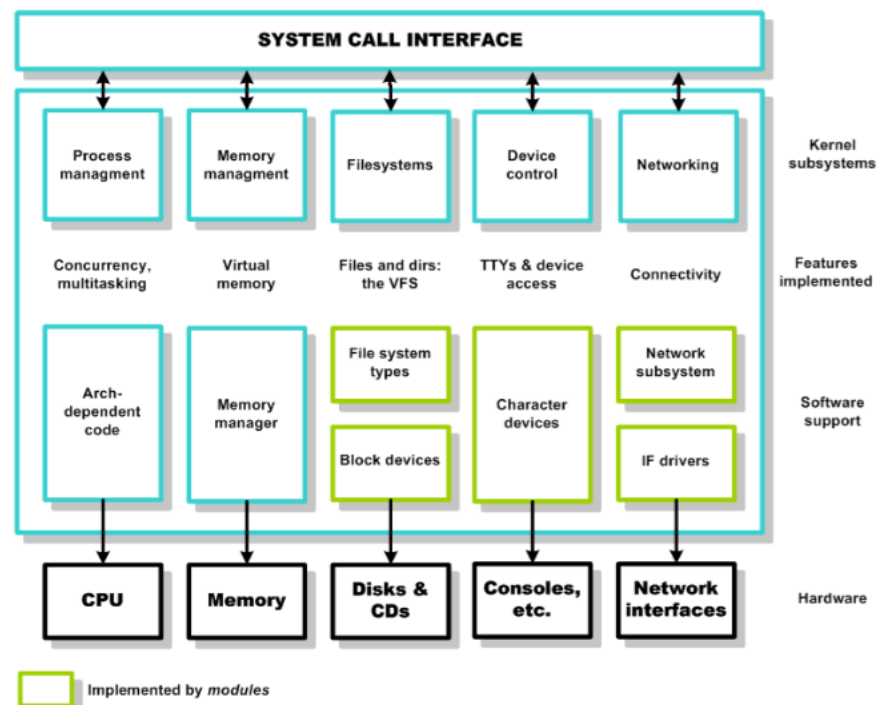
- Modern monolithic kernels use **loadable kernel modules** to provide extensibility during normal operation
 - Code runs in kernel mode



- Such kernels are called **modular kernels**
 - The term “monolithic kernel” refers primarily to how much OS code runs in kernel mode vs. user mode
 - By now, virtually all monolithic kernels are also modular

Modular kernels: Linux

- **Linux, in its core, is a monolithic kernel**
 - Executes in a single address space entirely in kernel mode
 - However, borrows much of the good from modular kernels



Modular kernels

- **Core mechanisms must remain resident; policies and optional drivers can be modular**
 - e.g., CPU scheduling, virtual memory management, inter-process communication are always compiled into the kernel
 - e.g., ATA drivers, SCSI drivers, filesystems commonly used with OS
 - Often a requirement to make the boot process more straightforward
- **Kernel defines interfaces for parts of kernel that require extensibility**
 - e.g., what set of operations must all filesystems support

Modular kernels

- **A modular kernel doesn't necessarily have a more well-defined structure**
 - It's just easier to extend the kernel's facilities
- **Kernel includes a module loader that can load and statically link a kernel module directly into the kernel**
 - At load time, module-references to kernel symbols are updated with actual addresses of kernel variables and functions
 - Module code is loaded/unloaded into/from kernel space

Modular kernels

■ Benefits

- Largely retains high performance of monolithic kernels
- Modules can control what symbols they expose, and encapsulate critical module state
- Yields a much smaller monolithic kernel, since only the modules required for proper functionality are loaded into memory

■ Drawbacks

- Only slightly reduces reliability issues!
- Modules run in kernel mode; bugs can still cause the OS to crashes

Microkernels

- **Minimal set of capabilities provided in kernel**
 - Restrict the kernel to contain only a minimal set of capabilities
- **Only ones that must use privileged CPU capabilities**
 - Process abstraction – context switches, CPU scheduling, interrupt handling, etc.
 - Memory abstraction – process address-space isolation, kernel/user memory separation
 - Inter-process communication facilities – required to allow user-mode processes to work together to implement system facilities

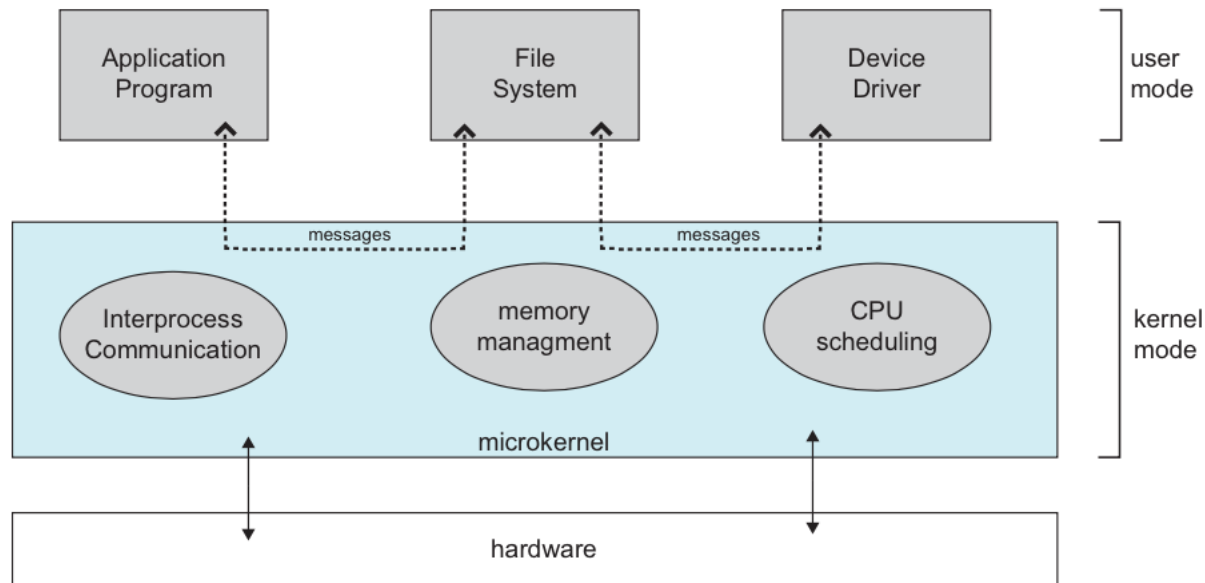
Microkernels

- **All other facilities are provided as user-mode programs**
 - Device drivers, filesystems, virtual memory pagers, etc.
- **Microkernels take “separation of policy and mechanism” approach to its furthest extent**
 - Generally, any component that implements a system policy should be implemented as a user-mode process
- **Some microkernels are extremely small**
 - E.g., less than 10K lines of code
 - Minix, by Andrew S. Tanenbaum, is a very well-known example

Microkernels

■ Direct functionality invocation is not possible

- An IPC mechanism must be implemented



Microkernels

- **E.g., use a text editor to create and save a file**
 - Text editor is a user-mode process
 - Filesystem implementation and disk driver are also user-mode processes

- **Filesystem service interacts with HDD driver via IPC to create the file on the physical disk**
 - When the text editor wants to create a file, it sends an IPC message to the filesystem service
 - When the file has been created (or in the event of failure), the filesystem service sends an IPC message back to the editor

Microkernels: L4

- **Jochen Liedtke began to study limitations of microkernels using inter-process communication**
 - Realized that the inter-process communication mechanism was doing a lot of unnecessary work
 - Implemented L3 and L4 microkernels with completely new IPC mechanisms
 - Achieved an order-of-magnitude performance improvement (!!!)
- **Liedtke's work greatly revived interest in microkernels**
- **Some of L4's IPC techniques reduce reliability benefits that microkernels are supposed to have**
 - But there are many contexts where this is actually OK!

Microkernels: Benefits

■ Very reliable

- A very small amount of kernel code to get correct
- Some are even formally proven correct against their specifications

■ If a system service crashes, it's a user-mode process

- It won't affect overall system stability. Just restart that process

■ System expansion is more accessible

- Microkernels are modular, and the various modules may be swapped, reloaded, and modified without affecting the kernel

Microkernels: Benefits

- **Supporting multiprocessor systems becomes very easy**
 - Just need to extend microkernel's IPC mechanism to support messages between processors
- **This capability made microkernels very interesting**
 - Companies were struggling to take earlier-generation single-CPU monolithic kernels and port them to multiprocessor systems
- **Can even implement non-local IPC facility with user-mode services**
 - Microkernel provides primitive IPC service between local processes
 - User-mode program provides higher-level IPC mechanism between local and/or remote processes across a network

Microkernels: Drawbacks

■ Microkernels have performance issues

- Important OS services live in user space
- Interactions between multiple services require context-switches

■ IPC through the kernel requires switching between kernel mode and user mode

- This IPC mechanism tends to add lots of overhead

■ Message-passing is asynchronous

- Most interactions between services are synchronous
- Process A makes a request to process B, and cannot progress until process B responds back to process A

Microkernels: Drawbacks

- **A simple request-response interaction requires four system calls**
 1. Send request
 2. Kernel mediation
 3. Receive reply
 4. Context switch back

Hybrid kernels

- **Hybrid kernels retain the same conceptual structure of microkernels, but move some services into the kernel**
 - Hybrid kernels move performance-critical services back into kernel space while preserving some modular and message-based abstractions
- **Some services continue to be provided by user-mode processes, allowing for easy extension of the OS**
 - Of course, they also lose the reliability benefits of microkernels
- **Offer flexibility in design, allowing for adaptation based on specific requirements and use cases**
 - They can achieve a balance between performance and modularity by combining the best aspects of monolithic and microkernel architectures

Hybrid kernels

- **However, hybrid kernels can be more complex to develop and maintain, possibly increasing development time and effort**
 - They may involve trade-offs between performance, modularity, and security, depending on the implementation

- **The flexibility of hybrid kernels can blur the lines between kernel and user mode components**
 - Potentially complicating system design and making it difficult to achieve an optimal balance

Hybrid kernels: Examples

■ Mac OS X's XNU kernel

- Based on CMU's Mach 3.0, with BSD facilities migrated into the kernel
- This combination created a flexible, high-performance foundation for Apple's operating systems

■ Windows NT

- Initially, was heavily influenced by the CMU's Mach project
- Unfortunately, performance wasn't satisfactory, so many services were migrated back into the kernel
- NT still runs several services as user-mode programs

Comparison table

Structure	Performance	Reliability	Modularity	Example
Simple	High	Very low	None	MS-DOS
Monolithic	Very high	Medium	Low	UNIX
Modular	High	Medium	Medium	Linux
Microkernel	Lower	High	High	Minix, L4
Hybrid	High	Medium-High	Medium-High	macOS, Windows NT

Summary

■ OSeS can be structured in several fundamental ways:

- Simple structures – Minimal separation between components; little or no protection
- Monolithic kernels – Most OS services execute in kernel mode within a single address space
- Modular kernels – Monolithic core extended through loadable kernel modules
- Microkernels – Minimal kernel; most services implemented as user-mode processes communicating via IPC
- Hybrid kernels – Combine microkernel concepts with performance-driven in-kernel services

Summary

- **Each approach represents a different balance between performance, reliability, modularity, maintainability, and security**
- **There is no universally “best” design**
 - Instead, OSeS make trade-offs based on target hardware, performance requirements, reliability constraints, extensibility goals
- **Modern OSeS rarely follow a single pattern strictly**
 - Most real-world systems blend elements from these core models, adapting structural decisions to practical constraints and evolving requirements
 - Understanding these patterns helps explain why OSeS differ — not only in implementation details, but in philosophy and design priorities